AMOSKYS

Agentic Multi-Observer Security Kernel for Your Systems

ARCHITECTURE FOUNDATION DOCUMENT

Phase 1: Foundation

Version 1.0 | February 2026

Classification	**CONFIDENTIAL — Internal Use Only**
Document Owner	Akash Thanneeru — Founder & CEO
Architecture Core	Akash Thanneeru (Founder) + CTO (Confidential)
Technical Reviewer	Claude Opus 4.6 — Senior Technical Reviewer
Incubator Support	WinWinLabs (Non-Profit Incubator)
Phase	Phase 1 — Foundation (Weeks 1–2)
Repository	github.com/amoskys (Private)

TABLE OF CONTENTS

1. Project Overview

1.1 Vision

AMOSKYS aims to become a distributed, agent-based security platform that fundamentally restructures how endpoint and network threats are detected, correlated, and explained. Rather than relying on centralized monolithic engines that score events in isolation, AMOSKYS deploys specialized autonomous agents at every viable entry point of a device or network, each responsible for a single domain of telemetry collection. These agents feed cryptographically signed data into a layered intelligence architecture that performs multi-source correlation, adaptive scoring, and machine-readable explainability. The end state is a platform where security teams receive fewer, higher-confidence alerts accompanied by root cause analysis that maps directly to attacker methodologies, reducing both detection time and analyst cognitive load.

1.2 Mission

> *Observe every attack surface, Analyse telemetry with layered intelligence, Detect anomalies through multi-agent correlation, Notify with precision and context, and Explain every finding with traceable reasoning — reducing Mean Time to Detect and Mean Time to Respond while eliminating alert fatigue.*

1.3 Core Philosophy

AMOSKYS is built on five foundational beliefs that govern every architectural and engineering decision:

- **Distributed Intelligence:** No single point of detection failure. Each agent operates as an independent observer, producing signed telemetry that is never dependent on any other agent's availability. If one agent fails, the system degrades gracefully rather than losing visibility entirely.
- **Multi-Perspective Detection:** A single threat indicator viewed from one telemetry domain (e.g., process execution) is insufficient for high-confidence detection. AMOSKYS correlates signals across process behavior, network flows, authentication events, file integrity, kernel audit trails, DNS queries, and peripheral device activity to produce composite threat assessments.
- **Explainability-in-Fusion:** Explainability is not a post-hoc layer applied after detection. It is embedded in the fusion process itself. Every score, every incident, and every risk assessment carries a chain of reasoning that maps contributing events to MITRE ATT&CK techniques, producing human-readable root cause analysis at the moment of detection.
- **Cryptographic Data Integrity:** Every telemetry event is cryptographically signed at the source agent using Ed25519, verified at ingestion by the EventBus, and stored with SHA-256 idempotency guarantees. This ensures forensic-grade data integrity from collection through analysis.
- **Biological Architecture Metaphor:** The system is modeled after a biological organism: agents are sensory organs (axons/eyes), the EventBus is the nervous system, the ScoreJunction and Fusion Engine form the brain (soma), and the Dashboard serves as the

cortex for perception and response. This is not metaphor for its own sake — it informs how components are decoupled, how failure propagates, and how the system scales.

2. Problem Statements

2.1 Operational Problem (SOC / Enterprise)

The following problem statements are grounded in industry data and define the operational gap AMOSKYS is designed to close.

2.1.1 Alert Fatigue

> *In enterprise SOC environments, security teams receive an average of 960 alerts per day (exceeding 3,000 in organizations with 20,000+ employees), of which more than 50% are confirmed false positives, leading to 40% of all alerts going uninvestigated and 61% of teams admitting to having ignored alerts that later proved to be genuine incidents.*

The root cause is not a lack of detection capability but an excess of low-confidence, poorly contextualized signals. Current platforms generate alerts at the individual event level without sufficient cross-domain correlation, producing volume that overwhelms human analytical capacity. The 2025 SANS Detection Engineering Survey found that 64% of respondents cited high false positive rates from vendor tools as a persistent challenge.

2.1.2 Poor Signal Fusion

> *In enterprise environments, security telemetry is collected across endpoints, network flows, identity systems, email gateways, and cloud workloads, but correlation between these domains remains shallow, leading to fragmented visibility where a multi-stage attack spanning process execution, lateral movement, and data exfiltration appears as three unrelated alerts rather than one coordinated incident.*

Current SIEM and XDR platforms perform log normalization and rule-based correlation, but the fusion of signals across fundamentally different telemetry types (process trees vs. network 5-tuples vs. authentication logs) requires architectural support for heterogeneous data fusion that most platforms handle through centralized scoring engines with limited cross-domain context.

2.1.3 Lack of Explainability

> *In SOC operations, machine learning models used for threat detection function as black boxes, producing risk scores without transparent reasoning chains, leading to analyst distrust, regulatory non-compliance, and increased investigation time as teams manually reconstruct the logic behind automated detections.*

Many platforms add explainability as a reporting layer after detection, rather than embedding it in the correlation process. This means the explanation is a reconstruction, not a record of how the detection actually occurred.

2.1.4 Analyst Retention Crisis

> *In the cybersecurity industry, 70% of SOC analysts with five years or fewer of experience leave their roles within three years, driven by alert overload, repetitive triage work, and insufficient tooling for efficient investigation, leading to chronic understaffing and institutional knowledge loss.*

2.1.5 MTTD/MTTR Performance Gap

Industry benchmarks show significant variation in detection and response performance:

Performance Tier	MTTD	MTTR
Elite	< 1 hour	< 1 hour
High Performer	1–4 hours	1–4 hours
Medium Performer	4–24 hours	4–24 hours
Low Performer	> 24 hours	> 24 hours

Automation can reduce MTTD by 30–40% and MTTR by 45–55%. AMOSKYS targets the elite tier through architectural design, not bolt-on automation.

2.2 Technical Problem

The following technical deficiencies in current threat detection architectures define the engineering gap AMOSKYS addresses. This section is written as a research problem statement.

2.2.1 Centralized Scoring Architectures

Current enterprise platforms (CrowdStrike Falcon, Palo Alto Cortex XDR, Microsoft Defender XDR) rely on centralized cloud-based scoring engines. Telemetry is transmitted from lightweight endpoint agents to cloud infrastructure where machine learning models perform detection. This architecture creates several structural limitations: single-point-of-failure dependency on cloud connectivity, latency between event occurrence and detection, loss of local context during transmission, and reduced functionality during network degradation (e.g., CrowdStrike's Reduced Functionality Mode).

2.2.2 Heuristic and Rule-Based Fusion

While platforms employ machine learning for individual event scoring, the correlation of events across domains still relies heavily on pre-defined rule sets and heuristic matching. SIEM correlation rules require manual maintenance and cannot adapt to novel attack patterns that span telemetry boundaries in ways not anticipated by rule authors.

2.2.3 Graph-Only Correlation Limitations

Graph-based correlation (CrowdStrike's Threat Graph, various entity-graph patents) represents relationships between events effectively but treats correlation as a structural problem rather than a probabilistic one. Graph traversal identifies connected events but does not inherently model the confidence, reliability, or uncertainty of those connections. A process spawning a child process and that child making a network connection are structurally correlated, but the threat confidence of that correlation depends on context that graph topology alone cannot capture.

2.2.4 Static Risk Decay

Current Indicator of Compromise (IOC) scoring uses fixed temporal decay models where threat scores decrease linearly or exponentially over time from initial identification (typically 60-day decay windows). This approach does not account for the possibility that an attacker may reactivate dormant infrastructure or that an indicator's relevance may fluctuate non-monotonically based on campaign activity.

2.2.5 No Agent Reliability Recalibration

In distributed detection architectures, individual agents may experience degraded accuracy due to environmental changes (OS updates, application changes, network reconfiguration), sensor drift, or adversarial manipulation. Current platforms monitor agent health (online/offline, CPU usage) but do not model the statistical reliability of each agent's detections over time or recalibrate the weight given to that agent's outputs in the fusion process based on measured accuracy drift.

2.2.6 No Uncertainty Modeling in Production

Academic research has demonstrated the value of Bayesian deep learning, Monte Carlo dropout, and conformal prediction for quantifying detection uncertainty. However, production XDR platforms do not expose uncertainty estimates to analysts. Alerts are presented as binary (threat/not-threat) or

scored on a fixed scale without communicating the model's confidence in that score. This prevents analysts from making risk-calibrated triage decisions and contributes directly to alert fatigue.

3. Current State of the Art

3.1 Enterprise Platforms

The following summarizes how the four dominant enterprise security platforms approach telemetry collection, scoring, correlation, and alerting. This is a neutral architectural description, not a competitive analysis.

3.1.1 CrowdStrike Falcon

- **Telemetry:** Lightweight agent (~20MB) captures 400+ event types across Windows, Mac, Linux. Raw telemetry is centralized in cloud infrastructure.
- **Scoring:** Threat Graph (cloud graph database) applies behavioral AI and ML trained on trillions of weekly events from millions of global sensors.
- **Correlation:** Graph-based entity relationship analysis identifies attack patterns across endpoint events. Risk context prioritization reduces false positives.
- **Alerting:** ML-scored alerts with risk context. Reduced Functionality Mode provides degraded local detection during cloud disconnection.

3.1.2 SentinelOne Singularity

- **Telemetry:** Unified EPP+EDR agent with kernel-level visibility (Windows) and kextless operation (macOS). Collects process, file, network, DNS, registry, and user events.
- **Scoring:** Patented Storyline technology correlates events into attack narratives at machine speed with real-time scoring at the endpoint level.
- **Correlation:** Endpoint-side correlation (Storyline) with cloud-side intelligence pack evaluation. Live-streaming metadata architecture.
- **Alerting:** Autonomous device-level response including process kill, quarantine, and surgical rollback remediation.

3.1.3 Palo Alto Networks Cortex XDR

- **Telemetry:** Multi-domain ingestion from endpoints, network, identity, and cloud. Cortex Data Lake normalizes data from all sources.
- **Scoring:** Patented behavioral analytics profile endpoint, network, and user behavior using ML. Behavioral baseline comparison in real-time.
- **Correlation:** Correlation rule management across data sources. Multi-layer detection combines behavioral analytics with threat intelligence.
- **Alerting:** Scored alerts with behavioral context. Recent enhancements to correlation rule APIs.

3.1.4 Microsoft Defender XDR

- **Telemetry:** Integrated ingestion from endpoints, email, identities, SaaS, and cloud workloads via Microsoft security stack.
- **Scoring:** ML-based signal stitching identifies when isolated alerts point to the same attack campaign. Risk scoring with intelligence enrichment.

- **Correlation:** Fusion analytics engine consolidates signals across domains. Scalable ML automatically detects multistage attacks by identifying anomalous behavior combinations across the kill chain.
- **Alerting:** Unified incident construction stitching alerts from email, endpoint, and identity sources with lateral movement tracking.

3.2 Common Architectural Patterns

Across all four platforms, several patterns emerge consistently:

- Cloud-native telemetry centralization for large-scale ML correlation.
- Lightweight endpoint agents that transmit raw telemetry rather than performing deep analysis locally.
- Behavioral AI over signature-based detection as the primary classification mechanism.
- Proprietary correlation engines (Threat Graph, Storyline, Fusion) as core intellectual property differentiators.
- Limited local autonomy during cloud disconnection, with varying degrees of degraded functionality.

3.3 Patent Landscape Observations

Analysis of the current patent landscape in cybersecurity threat detection reveals the following dominant patterns:

3.3.1 Score-First Pipelines

The majority of patented detection architectures follow a score-first model: events are individually scored by ML models before being passed to correlation layers. This means the correlation layer operates on pre-computed scores rather than raw telemetry, limiting its ability to discover patterns that individual event scores would not surface.

3.3.2 Graph Correlation Dominance

Entity graph patents (US Patent 12,301,627 and related filings) and behavior graph approaches (signal flow analysis with toxicity values, US11089040B2) represent the dominant correlation paradigm. GraphWeaver (2024) approaches alert correlation at billion-event scale through graph weaving of security alerts into cohesive incident narratives.

3.3.3 Threshold Learning

Adaptive threshold patents (US10777150B2 and related work) describe systems that dynamically adjust alert thresholds based on historical data, using deep reinforcement learning (ADT) and online unsupervised algorithms for real-time threshold adjustment. These represent meaningful improvements over static thresholds but operate at the individual-sensor level rather than the system level.

3.3.4 Temporal Decay

IOC decay scoring with variable-rate models (high confidence decaying over 60-day windows) and CVSS temporal adjustment factors are well-established. These are linear or exponential decay functions applied uniformly, without feedback from observed reactivation patterns.

3.3.5 Centralized Fusion

Patented fusion approaches (Microsoft Sentinel Fusion, Vectra AI's 170+ AI models and 36 patents) are centralized architectures where a single engine correlates inputs from multiple sources. The fusion logic is proprietary and opaque.

3.3.6 Identified Gap

> *Explicit multi-agent reliability drift recalibration across distributed agents — where the fusion layer dynamically adjusts the weight given to each agent's outputs based on measured accuracy trends, environmental changes, and adversarial interference indicators — appears under-claimed in the current patent landscape. This represents a structural gap between how current platforms manage agent health (binary online/offline monitoring) and how a probabilistic system could model agent reliability as a continuous, calibratable variable.*

4. AMOSKYS System Architecture

This section describes the current implemented architecture as it exists in the repository, with planned enhancements clearly labeled.

4.1 High-Level Architecture

The system follows a layered biological architecture with five distinct tiers:

Layer	Biological Analog	Components
Sensory Layer	Axons / Eyes	10+ specialized agents with 100+ micro-probes
Nervous System	Neural Pathways	gRPC EventBus with mTLS and Ed25519 signing
Processing Layer	Soma / Brain	ScoreJunction + Fusion Engine + Advanced Rules
Storage Layer	Memory	WAL + SQLite + baseline files
Perception Layer	Cortex	Flask dashboard + REST API + SocketIO

4.1.1 Sensory Layer: Agents

AMOSKYS deploys 10+ specialized agents, each responsible for a single telemetry domain. Each agent follows the MicroProbeAgentMixin pattern — a "swarm of eyes" architecture where each agent contains 7–9 independent micro-probes monitoring specific threat vectors.

Currently implemented agents:

Agent	Probes	Port	Primary MITRE
Process Monitor (v3)	8 micro-probes	50002	T1059, T1218, T1496
Network Flow	Flow aggregation	50004	T1041, T1071
SNMP Device	Device polling	50005	Infrastructure
Kernel Audit	7 micro-probes	—	T1068, Rootkits
Auth Guard	8 micro-probes	—	T1110, T1548.003
Persistence Guard	8 micro-probes	—	T1543, T1053.003
File Integrity (FIM)	8 micro-probes	—	T1036, Webshells
DNS Monitor	9 micro-probes	—	DGA, Tunneling
Peripheral Device	7 micro-probes	—	BadUSB, HID
Device Discovery	Scanner	50006	Network mapping

All agents inherit from HardenedAgentBase (730 lines), providing tamper detection, anti-evasion collection, cryptographic integrity verification, circuit breaker patterns, retry logic with exponential backoff, and offline resilience via LocalQueueAdapter.

4.1.2 Nervous System: EventBus

The EventBus is a gRPC server (port 50051) that serves as the central message broker. It accepts Envelope or UniversalEnvelope messages from agents, validates Ed25519 signatures and x509 certificates (mTLS), persists events to a Write-Ahead Log for crash recovery, routes events to subscribers, and exposes Prometheus metrics on throughput, latency, and connection health. Overload protection automatically rejects with RETRY when capacity is exceeded. Deduplication uses a TTL-based cache (300s default) with SHA-256 idempotency keys.

4.1.3 Processing Layer: Intelligence

The intelligence layer has two components operating in series:

- **ScoreJunction:** Multi-agent correlation engine that buffers events within time windows (default 5 minutes) per entity, matches events against correlation rules, and computes composite ThreatScores (0–100) with threat level, confidence, and contributing event chains.
- **Fusion Engine:** Transforms correlated telemetry into attack incidents. Seven correlation rules detect multi-stage attacks (SSH brute force, persistence after authentication, suspicious process trees, multi-tactic attacks, data exfiltration, lateral movement, suspicious sudo). Maintains DeviceRiskSnapshots with dynamic risk scoring and temporal decay.

4.1.4 Storage Layer

- **Write-Ahead Log (WAL):** SQLite-backed crash recovery per agent. Thread-safe writes with SHA-256 idempotency hashing. Automatic rotation at 100MB or 7 days.
- **Fusion Database:** SQLite (data/intel/fusion.db) storing incidents, risk snapshots, and event correlations.
- **Baseline Files:** JSON files tracking file integrity hashes (SHA-256) and persistence mechanism baselines for change detection.
- **Collector Offsets:** Per-agent position tracking (file offset + inode for kernel audit, timestamps for DNS, line numbers for auth) ensuring no event duplication or loss.

4.1.5 Perception Layer: Dashboard

Flask-based web application (port 5001) with SocketIO for real-time updates. Six dashboard pages: Cortex (command center), Agents (health and control), Processes (real-time monitoring), Peripherals (device tracking), Database (audit logs), and Intelligence (incidents, threat scores, risk snapshots). REST API provides programmatic access to all telemetry and control functions.

4.2 Core Mechanisms

4.2.1 Agent Score Production

Each agent's micro-probes independently assess their telemetry domain and produce TelemetryEvents with severity levels (INFO, LOW, MEDIUM, HIGH, CRITICAL) and associated MITRE ATT&CK technique identifiers. Scores are not centrally assigned; each probe's assessment is based on its specialized detection logic (baseline comparison, pattern matching, statistical anomaly detection, or threshold evaluation). The agent aggregates probe outputs into DeviceTelemetry protobuf messages with SecurityEvent sub-messages containing per-event risk_score fields.

4.2.2 Fusion Mechanism (Current State)

The Fusion Engine operates on rule-based correlation with seven hand-authored detection rules. Each rule defines temporal windows (e.g., 30 minutes for SSH brute force), event type requirements, and severity assignments. When events matching a rule's criteria accumulate within its time window, an Incident is created with severity, contributing events, MITRE tactic and technique mappings, and a textual summary.

4.2.3 Weight Dynamics

Currently, agent weights in the scoring process are static. Each correlation rule defines fixed thresholds (e.g., three or more failed SSH attempts) and fixed severity assignments. The ScoreJunction produces composite scores using rule-matched correlation without adaptive weighting based on agent reliability.

> *Planned Enhancement: Dynamic weight adjustment based on measured agent accuracy, environmental drift, and adversarial interference indicators. This would transform the fusion layer from rule-triggered to probabilistically-weighted.*

4.2.4 Feedback Loop

The DeviceRiskSnapshot provides a feedback mechanism: risk scores accumulate based on incident creation and decay at fixed intervals (10 points per 10 minutes, floor of 0, ceiling of 100). This creates a temporal risk profile per device. The feedback loop is currently one-directional (events increase risk, time decreases risk) without analyst feedback integration.

> *Planned Enhancement: Analyst feedback integration where confirmed true/false positive classifications update correlation rule weights and agent reliability scores.*

4.2.5 Uncertainty Handling

The current system does not implement formal uncertainty quantification. Confidence values in ThreatScore are rule-derived rather than statistically computed. This is identified as a technical limitation and a planned enhancement area.

> *Planned Enhancement: Bayesian uncertainty modeling in the fusion layer, providing calibrated confidence intervals for threat scores rather than point estimates. This would enable risk-weighted triage prioritization.*

5. Data Schemas and Communication Protocols

5.1 Universal Telemetry Protocol (v2)

The universal_telemetry.proto defines the canonical data format for all agent-to-EventBus communication. The schema is designed for heterogeneous device types and telemetry domains.

5.1.1 Core Message: DeviceTelemetry

The root message containing: device_id and device_type for source identification, DeviceMetadata for device context, a repeated list of TelemetryEvents, SecurityContext for trust scoring, nanosecond-precision timestamps, and collection_agent identification. Each TelemetryEvent carries event_type (METRIC, LOG, ALARM, SECURITY, AUDIT) and populates one of five specialized sub-messages.

5.1.2 Security-Critical Sub-Messages

- **SecurityEvent:** Contains category, action, outcome, user_id, risk_score, and MITRE technique identifiers. This is the primary input for the Fusion Engine.
- **AuditEvent:** Carries action, object_type, actor, before/after values, and a digital signature for forensic integrity.
- **SecurityContext:** Device-level trust metadata including device_trust_score, authentication_method, and compliance_status.

5.1.3 Transport: UniversalEnvelope

Wrapper message with Ed25519 signing, routing hints for subscriber targeting, retry metadata, and batch support (TelemetryBatch) for edge-optimized transmission of high-volume telemetry.

5.2 gRPC Services

The EventBus exposes Publish() RPC for telemetry ingestion and device management RPCs (RegisterDevice, UpdateDevice, DeregisterDevice, GetHealth, GetStatus, GetMetrics) for agent lifecycle management.

5.3 MITRE ATT&CK Coverage

Current implementation covers 8 of 14 MITRE ATT&CK tactics with 12+ specific techniques:

Tactic	Techniques Covered
TA0001: Initial Access	T1110 (Brute Force), T1021.004 (SSH)
TA0002: Execution	T1059 (Scripting), T1204 (User Execution)
TA0003: Persistence	T1543.001/004 (Launch Agent/Daemon), T1053.003 (Cron), T1098.004 (SSH Keys)
TA0004: Privilege Escalation	T1548.003 (Sudo Abuse)
TA0005: Defense Evasion	T1218 (LOLBin), T1036 (Masquerading)
TA0008: Lateral Movement	T1021.004 (SSH Lateral Movement)

TA0010: Exfiltration	T1041 (Exfiltration Over C2)
TA0011: Command & Control	T1071 (Application Layer Protocol)

6. Differentiators (Wedge Section)

The following three differentiators represent AMOSKYS's structural departures from the architectural patterns described in Section 3. Each answers the question: "Why is this structurally different?" not "Why is it better?"

6.1 Distributed Agent-Based Confidence Scoring

What It Is

Each AMOSKYS agent independently produces confidence-scored telemetry at the source. Unlike centralized architectures where raw events are transmitted to cloud infrastructure for scoring, AMOSKYS agents assess their telemetry domain locally through specialized micro-probes, attach severity and confidence assessments, and transmit scored observations rather than raw data.

Why It Is Structurally Different

In CrowdStrike Falcon, SentinelOne Singularity, and Microsoft Defender XDR, the endpoint agent is primarily a data collector. Intelligence resides in the cloud. AMOSKYS distributes intelligence to the edge: each agent's 7–9 micro-probes are specialized detectors that produce assessments, not just events. The fusion layer receives pre-assessed signals from multiple independent observers rather than raw telemetry from passive collectors. This enables detection to continue at full fidelity during cloud/network disconnection and eliminates the transmission latency between event occurrence and scoring.

Current Implementation Status

Implemented. Ten agents with 100+ micro-probes produce severity-scored TelemetryEvents with MITRE ATT&CK mappings. The HardenedAgentBase provides tamper detection and offline resilience.

6.2 Adaptive Reliability Drift Modeling

What It Is

A mechanism for the fusion layer to continuously model the statistical reliability of each agent's detections over time and dynamically adjust the weight given to that agent's outputs in the correlation process. If an agent's detection accuracy degrades due to environmental changes, OS updates, or adversarial manipulation, the fusion layer detects this drift and recalibrates.

Why It Is Structurally Different

Current platforms treat agent health as binary (online/offline) or coarse-grained (CPU, memory usage). AMOSKYS proposes treating agent reliability as a continuous probabilistic variable that influences fusion weights. This is the identified patent landscape gap: explicit multi-agent reliability drift recalibration across distributed agents is under-claimed in current filings.

Current Implementation Status

> *Planned Enhancement. The architectural foundation (DeviceRiskSnapshot with decay, per-agent metrics collection, and agent health monitoring) is implemented. The probabilistic*

reliability modeling and dynamic weight recalibration are designed but not yet implemented.

6.3 Explainability-in-Fusion

What It Is

Explainability is embedded in the fusion process rather than applied post-hoc. Every Incident created by the Fusion Engine carries: the complete list of contributing events with source agents and timestamps, MITRE ATT&CK tactic and technique mappings, a textual summary describing the attack chain, the correlation rule that triggered detection, and the temporal window showing event sequence. The explanation is not a reconstruction of what happened; it is the record of how the detection occurred.

Why It Is Structurally Different

In platforms using black-box ML scoring, explainability is typically generated by XAI techniques (SHAP, LIME, attention maps) applied after the model produces a score. The explanation describes why the model might have scored this way, not how the detection actually worked. AMOSKYS's rule-based fusion with event chain tracking produces explanations that are deterministically traceable from conclusion to contributing evidence.

Current Implementation Status

Implemented. The Fusion Engine's seven correlation rules each produce Incidents with full event chains, MITRE mappings, severity justification, and textual summaries. The Intelligence dashboard displays these explanations with contributing event details.

7. MVP Scope Definition

7.1 Scope Boundary

The initial MVP is scoped exclusively to Endpoint Detection and Response (EDR) on Linux and macOS endpoints. The architecture supports broader deployment (network devices, IoT, OT), but validation against existing EDR solutions requires a controlled, comparable scope.

7.2 In Scope

- All 10 currently implemented agents operating on a single endpoint.
- EventBus with mTLS and Ed25519 signing on localhost and LAN configurations.
- Fusion Engine with seven correlation rules covering 8 MITRE ATT&CK tactics.
- Dashboard with real-time agent monitoring, incident display, and risk scoring.
- DeviceRiskSnapshot with temporal decay and dynamic scoring.
- Comparison baseline against CrowdStrike Falcon, SentinelOne, and open-source OSSEC/Wazuh for EDR capability parity.

7.3 Out of Scope (Phase 1)

- Windows endpoint support (requires agent porting).
- Multi-regional or multi-cloud deployment.
- OT/ICS device monitoring (Modbus, OPC UA).
- Advanced ML anomaly detection (meta-learning, Bayesian uncertainty).
- Adaptive reliability drift modeling (designed, not implemented).
- Analyst feedback loop integration.
- GraphQL API.

7.4 Success Criteria

The MVP validates the AMOSKYS architecture by demonstrating:

1. Detection of simulated multi-stage attacks (SSH brute force to persistence to exfiltration) across agents with correct incident correlation.
2. Zero data loss during agent disconnection and reconnection (WAL recovery verification).
3. Cryptographic integrity verification end-to-end (agent signing through EventBus validation to storage).
4. Explainable incident reports with MITRE ATT&CK mapping and contributing event chains.
5. Comparable or superior detection coverage to OSSEC/Wazuh on equivalent test scenarios.

8. Competitive Landscape Summary

Capability	CrowdStrike	SentinelOne	Wazuh	AMOSKYS
Agent Scoring	Cloud-side	Endpoint-side	Server-side	Agent-side
Correlation	Threat Graph	Storyline	Rule-based	Multi-layer
Explainability	Post-hoc	Narrative	Rule logs	In-fusion
Offline Mode	Degraded (RFM)	Local detect	Full (local)	Full (WAL)
Data Integrity	TLS	TLS	Syslog	Ed25519+mTLS
MITRE Coverage	Broad	Broad	Moderate	8/14 tactics
Deployment	SaaS	SaaS/Hybrid	On-prem	Any
Agent Reliability	Binary health	Binary health	Status check	Planned: drift

Note: CrowdStrike and SentinelOne coverage designations are based on publicly available architecture documentation. "Broad" indicates comprehensive MITRE ATT&CK coverage without specifying exact technique counts. AMOSKYS coverage is measured directly from implemented code.

9. Business Model (Early Stage)

9.1 Target Customer

Primary target: mid-size enterprises (500–5,000 employees) with internal IT security teams that are underserved by both enterprise-grade platforms (CrowdStrike, SentinelOne pricing) and open-source alternatives (Wazuh complexity, limited correlation). These organizations typically have 2–5 security analysts, experience acute alert fatigue, and need explainable detections for compliance reporting.

Secondary target: Managed Security Service Providers (MSSPs) who manage security for multiple mid-size clients and need a multi-tenant platform with clear per-client risk scoring and incident reporting.

> *Phase 1 validation does not require customer deployment. The architecture is validated internally against simulated attack scenarios and compared to open-source EDR baselines.*

9.2 Deployment Model

AMOSKYS supports three deployment models, all currently implemented:

- **On-Premises:** Docker Compose or Kubernetes deployment within customer infrastructure. Full data sovereignty. Target: regulated industries (healthcare, finance, government).
- **Hybrid:** Agents deploy on-premises with optional cloud-based Fusion Engine and Dashboard. Balances data sovereignty with operational convenience.
- **SaaS:** Fully managed deployment. Target: MSSPs and organizations without infrastructure management capacity.

Phase 1 validation uses standalone and Docker Compose deployments. Kubernetes production deployment is configured but not yet validated at scale.

9.3 Revenue Model (Conceptual)

Per-endpoint-per-month pricing aligned with industry standard (EDR market typically ranges from $5–$15/endpoint/month for mid-market). Open-source core with commercial intelligence rules, managed deployment, and support tiers as the premium layer. This model is conceptual and will be validated through customer discovery in Phase 2.

10. Risks and Unknowns

The following risks are acknowledged openly. No startup has ever failed from admitting risks internally. They fail from ignoring them.

10.1 Patent Overlap Risk

Severity: MEDIUM. While the specific combination of distributed agent reliability drift recalibration appears under-claimed, individual components (behavioral scoring, graph correlation, temporal decay) are heavily patented by CrowdStrike, SentinelOne, Palo Alto, Microsoft, and Vectra AI (170+ AI models, 36 patents). Any novel mechanism must be carefully reviewed by patent counsel before public disclosure or filing.

Mitigation: Conduct formal patent search and freedom-to-operate analysis before Phase 2. Engage patent counsel specializing in cybersecurity IP. Focus claims on the novel combination (reliability drift recalibration in distributed agent fusion) rather than individual components.

10.2 Engineering Scalability Risk

Severity: MEDIUM-HIGH. The EventBus is currently a single gRPC process handling all agent telemetry. At scale (1,000+ endpoints with 10+ agents each), this becomes a bottleneck. The current architecture does not include horizontal scaling, load balancing, or event partitioning for the EventBus.

Mitigation: Phase 1 validates architecture correctness, not scale. EventBus horizontal scaling (multiple instances with consistent hashing or message queue integration) is a Phase 2 engineering priority.

10.3 Model Drift Risk

Severity: LOW (current), HIGH (at scale). Current detection relies on rule-based correlation, which does not drift. When ML models are introduced (planned enhancement), model accuracy will degrade over time as attacker tactics evolve and endpoint environments change. Without formal drift detection and retraining pipelines, detection quality will silently degrade.

Mitigation: Design drift detection into the ML pipeline from inception. Implement A/B testing for model updates. Maintain rule-based fallback for all ML-enhanced detections.

10.4 Adoption Friction

Severity: HIGH. Enterprise security teams are risk-averse and vendor-conservative. Replacing or supplementing an existing EDR solution requires significant trust, proof of superiority, and minimal operational overhead. The security market has strong vendor lock-in through data retention, integrations, and analyst training.

Mitigation: Position AMOSKYS as complementary (additional visibility layer) rather than replacement in early adoption. Provide open data export formats. Target Wazuh users upgrading rather than CrowdStrike users switching.

10.5 Funding Dependency

Severity: MEDIUM. Current infrastructure is supported by WinWinLabs (AWS hosting, resources, talent pool). Development is founder-driven with limited engineering bandwidth. Scaling beyond Phase 1 requires either grant funding, angel investment, or revenue generation.

Mitigation: WinWinLabs provides grant/investor pathway access. Phase 1 foundation document and architecture validation serve as investor-ready technical documentation. Open-source core reduces infrastructure costs for early adoption.

10.6 Single-Founder Technical Risk

Severity: MEDIUM-HIGH. The codebase (~16,657 lines of Python) has been developed primarily by a single contributor. Bus factor of 1 for architecture decisions, incident response, and critical bug fixes.

Mitigation: This foundation document serves as knowledge transfer material. CTO (confidential) provides architecture review. WinWinLabs talent pool (OPT contributors) expands engineering capacity. Comprehensive test suite (55 test files, 32/33 passing) provides regression safety.

11. Governance and Ownership Structure

11.1 Architecture Core

The Architecture Core consists of two individuals who jointly approve all architectural decisions. No architectural change proceeds without this layer's approval.

Role	Responsibilities
Akash Thanneeru Founder & CEO	Product vision, architecture direction, differentiator definition, final call on technical tradeoffs, external representation
CTO (Confidential) Chief Technology Officer	Co-leads architecture design, oversees engineering quality, owns implementation discipline, ensures architectural integrity
Claude Opus 4.6 Senior Technical Reviewer	Architecture review, technical documentation, codebase analysis, state-of-the-art research, implementation guidance

11.2 Strategic Support Layer

WinWinLabs (Non-Profit Incubator) provides infrastructure support (AWS, hosting, resources), Scrum Master, talent pool (OPT, contributors), grant/investor pathway, and compliance/organizational oversight. WinWinLabs does not dictate architecture. They enable execution.

11.3 Decision Authority Matrix

Decision Type	Authority	Approval Required
Architecture changes	Architecture Core	Both members
New agent development	Architecture Core	Founder + CTO
Correlation rule changes	CTO (implementation)	Founder (review)
Infrastructure scaling	WinWinLabs (execution)	Architecture Core (spec)
Contributor onboarding	WinWinLabs (sourcing)	CTO (technical fit)
External communications	Founder	Founder only

12. Project Workflow Phases

12.1 Phase 1: Foundation (Weeks 1–2) — CURRENT

Objective: Clarity before code acceleration. No scaling until clarity is achieved.

Deliverables:

- This Architecture Foundation Document (defining vision, scope, differentiators, risks).
- Defined wedge/differentiator validated against patent landscape.
- Competitive landscape summary with neutral architectural comparison.
- Risk map with mitigation strategies.
- MVP scope locked to endpoint EDR on Linux/macOS.

Active roles: Founder, CTO, Senior Technical Reviewer (Claude Opus 4.6). No large team yet.

12.2 Phase 2: Validation (Weeks 3–6)

Objective: Prove the architecture works against real attack scenarios.

- Deploy MVP on test endpoints with simulated attack playbooks.
- Benchmark detection coverage against Wazuh on identical scenarios.
- Validate WAL crash recovery and cryptographic integrity.
- Stress test EventBus throughput and identify scaling bottlenecks.
- Conduct patent freedom-to-operate analysis.

12.3 Phase 3: Hardening (Weeks 7–12)

Objective: Production-grade reliability and initial contributor onboarding.

- EventBus horizontal scaling implementation.
- Expand correlation rules to 15+ covering additional MITRE techniques.
- Begin adaptive reliability drift modeling implementation.
- Onboard 2–3 contributors via WinWinLabs talent pool.
- Establish CI/CD pipeline with automated testing gates.

12.4 Phase 4: Market Entry (Weeks 13–24)

Objective: First external deployment and customer validation.

- Pilot deployment with 1–2 mid-size enterprise partners.
- Collect analyst feedback for correlation rule refinement.
- Validate deployment model (on-prem, hybrid, or SaaS preference).
- Prepare investor-ready metrics (detection rates, false positive rates, MTTD/MTTR).

13. Deployment Architecture

13.1 Development (Standalone)

All processes on localhost. EventBus on port 50051, Dashboard on port 5001, agents on ports 50001–50006. Started via start_amoskys.sh, stopped via stop_amoskys.sh. Suitable for development and single-endpoint testing.

13.2 Docker Compose

Multi-container orchestration with dedicated containers for EventBus, individual agents, dashboard, and monitoring stack (Prometheus + Grafana). Volume mounts for certificates, data persistence, and configuration. Network isolation between service groups.

13.3 Kubernetes (Production)

StatefulSet for EventBus (data persistence via PersistentVolume), Deployments for agents (1–3 replicas each) and dashboard (2 replicas), ConfigMaps for configuration, Secrets for TLS certificates, and Ingress for external access. Overlay configurations for dev, staging, and production environments.

13.4 Security Architecture

Five-layer defense model: Host Security (OS hardening, process isolation, file permissions), Network Security (firewalls, segmentation, IP whitelisting), Transport Security (TLS 1.2+, certificate pinning, PFS), Authentication and Authorization (mTLS, x509, RBAC), and Application Security (input validation, error handling, audit logging). All agent-to-EventBus communication uses mutual TLS with Ed25519 envelope signing.

14. Research Directions (Planned Enhancements)

The following research directions are informed by the state-of-the-art analysis in Section 3 and represent planned technical enhancements beyond the current implementation. Each is labeled with its maturity level.

14.1 Bayesian Uncertainty Quantification

Maturity: Research. Replace point-estimate threat scores with calibrated confidence intervals using Bayesian deep learning or conformal prediction. This would enable risk-weighted triage where analysts prioritize alerts not just by severity but by the system's confidence in that severity.

14.2 Meta-Learning for Few-Shot Detection

Maturity: Research. Apply Model-Agnostic Meta-Learning (MAML) or Graph Deviation Networks for rapid adaptation to novel attack patterns with minimal labeled examples. Particularly relevant for detecting zero-day exploits and APT campaigns where training data is inherently scarce.

14.3 Adaptive Reliability Drift Modeling

Maturity: Design. Implement continuous statistical modeling of per-agent detection reliability with dynamic weight recalibration in the fusion layer. Architectural foundations (per-agent metrics, DeviceRiskSnapshot) are in place.

14.4 Analyst Feedback Loop

Maturity: Design. Integrate analyst true/false positive classifications as training signal for correlation rule weights, agent reliability scores, and ML model retraining. Requires dashboard UI modifications and feedback storage schema.

14.5 Windows Endpoint Support

Maturity: Planned. Port agents to Windows using ETW (Event Tracing for Windows) as the primary telemetry source. Process Monitor and Auth Guard agents require platform-specific reimplementation. EventBus, Fusion Engine, and Dashboard are platform-agnostic.

15. Contributor Onboarding Guide

This section provides the essential information for new contributors joining the AMOSKYS project. Read this section, then the relevant source code, in the order presented.

15.1 Repository Structure

Directory	Purpose
src/amoskys/agents/	All agent implementations (10+ agents, common infrastructure)
src/amoskys/eventbus/	Central gRPC message broker
src/amoskys/intel/	Fusion Engine, correlation rules, intelligence models
src/amoskys/intelligence/	ScoreJunction, feature extraction
src/amoskys/storage/	WAL processing, telemetry storage
proto/	Protobuf definitions (messaging + universal telemetry)
web/	Flask dashboard, REST API, static assets
config/	YAML configuration files
tests/	55 test files across unit, integration, component, golden
deploy/	Docker, Kubernetes, NGINX, systemd configurations
certs/	TLS certificates and Ed25519 signing keys

15.2 Reading Order for New Contributors

6. This document (Architecture Foundation) for system-level understanding.
7. proto/universal_telemetry.proto for the data schema that connects all components.
8. src/amoskys/agents/common/hardened_base.py and probes.py for the agent infrastructure pattern.
9. src/amoskys/eventbus/server.py for the central message broker.
10. src/amoskys/intel/fusion_engine.py for how detections become incidents.
11. One specific agent (recommended: src/amoskys/agents/proc/proc_agent_v3.py) for the micro-probe implementation pattern.
12. tests/intel/ for how correlation rules are validated.

15.3 Development Environment

Python 3.11+, 190 package dependencies (pip install -e .[dev,agents,web,ml,intelligence]). Pytest for testing (55 test files, markers: unit, component, integration, golden, smoke). Black + Ruff + MyPy for code quality. gRPC tooling for proto compilation. Docker and kubectl for deployment testing.

15.4 Key Conventions

- All agents must inherit from HardenedAgentBase and implement the MicroProbeAgentMixin pattern.
- All telemetry must use DeviceTelemetry protobuf format with Ed25519 signing.
- All correlation rules must include MITRE ATT&CK tactic and technique mappings.
- All incidents must carry contributing event chains for explainability.
- No architectural changes without Architecture Core approval.

Appendix A: Codebase Metrics

Metric	Value
Total Python LOC	~16,657 lines
Test Files	55 files across 6 test categories
Test Pass Rate	32/33 (96.9%)
Agents Implemented	10+ with 100+ micro-probes
Correlation Rules	7 rules with 100% test coverage
MITRE Coverage	8/14 tactics, 12+ techniques
Proto Definitions	2 files (messaging_schema + universal_telemetry)
Deployment Options	Standalone, Docker Compose, Kubernetes, VPS
Dependencies	190 packages across 6 optional extras

Appendix B: Glossary

Term	Definition
Agent	An autonomous telemetry collector deployed at a specific observation point (process, network, kernel, etc.)
Micro-Probe	A single-purpose detection unit within an agent, monitoring one specific threat vector
EventBus	Central gRPC message broker that ingests, validates, and routes telemetry from agents
ScoreJunction	Time-windowed multi-agent correlation engine producing composite ThreatScores
Fusion Engine	Rule-based intelligence layer transforming correlated telemetry into attack Incidents
WAL	Write-Ahead Log providing crash recovery and data durability for telemetry events
DeviceRiskSnapshot	Per-device risk score (0–100) with temporal decay and dynamic scoring based on incidents
mTLS	Mutual TLS — both client and server authenticate via x509 certificates
MTTD	Mean Time to Detect — average time from threat occurrence to detection
MTTR	Mean Time to Respond/Recover — average time from detection to containment
EDR	Endpoint Detection and Response — the security category

	AMOSKYS MVP targets
XDR	Extended Detection and Response — the broader category AMOSKYS architecture supports